

Apuntes Completos y Detallados: Programacion Orientada a Objetos (POO) en Java

UNIDAD 5: Conceptos Basicos de la POO

La POO es un paradigma que cambia el enfoque "estructurado" (codigo espagueti donde variables y funciones estan separadas) por uno donde modelamos el software basandonos en cosas de la vida real. Empaquetamos datos y logica juntos en unidades llamadas **Objetos**.

1. La Triada del Objeto (Conceptos Fundamentales)

Para que algo sea considerado un objeto, debe tener 3 cosas:

1. **Identidad:** Lo que permite distinguir un objeto de otro en memoria, aunque sus datos sean identicos (ej. dos monedas de 1 euro son iguales, pero son dos objetos fisicos distintos).
2. **Estado (Atributos):** Los datos o caracteristicas en un momento dado (ej. color, vida, nivel).
3. **Comportamiento (Metodos):** Las acciones o respuestas que el objeto puede dar (ej. frenar(), saludar()).

2. Clase vs. Objeto (Abstraccion)

El proceso de ignorar detalles irrelevantes y quedarse con lo importante se llama **abstraccion**.

- **La Clase (El Plano):** Es la definicion abstracta, la plantilla. Define que habra, pero no ocupa memoria con datos reales. Nadie puede "vivir" en un plano.
- **El Objeto (La Instancia):** Es la concrecion fisica en la memoria. Se crea usando la palabra "new" a partir de la clase (es la "casa construida").

```
// DEFINICION DE LA CLASE (El molde)
public class Personaje {
    String nombre;
    int vida;
}

// CREACION DE OBJETOS (En el main)
Personaje heroe = new Personaje(); // Instancia 1
heroe.nombre = "Aragorn";

Personaje villano = new Personaje(); // Instancia 2 (independiente de la 1)
villano.nombre = "Orco";
```

3. Miembros de una Clase y el Puntero "this"

Las clases tienen Atributos y Metodos. Cuando un metodo recibe un parametro que se llama *exactamente igual* que un atributo de la clase, Java se confunde (Shadowing o Sombreado). Para evitarlo, usamos **this**, que significa "MI atributo" (el del objeto actual que ejecuta el codigo).

```
public void cambiarNombre(String nombre) {
    // this.nombre -> El atributo general de la clase
    // nombre -> El parametro temporal que nos acaban de pasar
    this.nombre = nombre;
}
```

4. Encapsulamiento y Visibilidad (La Caja Negra)

Consiste en ocultar los detalles internos de un objeto para proteger su integridad (ej. que nadie le ponga vida negativa a un personaje desde fuera).

REGLA DE ORO: Por defecto, los atributos deben ser SIEMPRE private.

- **private:** Acceso restringido. Solo la propia clase puede tocarlo.
- **public:** Acceso total. La "puerta abierta".
- **Getters y Setters:** Son la "Interfaz Publica" (metodos public) que permiten leer (Getter) o modificar (Setter) un atributo private de forma controlada y segura.
- **Metodos Helpers:** Metodos private internos que hacen calculos complejos (ej. calcular si un golpe es critico) y que el exterior no necesita ni debe ver.

```
public class Personaje {
    private int vida; // Protegido (nadie puede hacer: p.vida = -100)

    public int getVida() { return this.vida; } // Getter

    public void setVida(int vida) { // Setter inteligente
        if (vida < 0) {
            System.out.println("Error: Vida negativa");
            this.vida = 0; // Corrige el error internamente
        } else {
            this.vida = vida;
        }
    }
}
```

5. Constructores: Naciendo con Estado

Son metodos especiales que se ejecutan automaticamente al usar "new". Su mision es garantizar que el objeto nazca en un estado valido (que no haya objetos "zombies" con datos vacios).

- **Reglas:** Tienen exactamente el *mismo nombre que la clase* y *no tienen tipo de retorno* (ni void).

IMPORTANTE: Si tu NO escribes un constructor, Java te regala uno invisible vacio (public Personaje() {}). PERO, en el momento en que tu escribas un constructor propio con parametros, Java retira el por defecto.

Tipos de constructores (Sobrecarga):

```
public class Personaje {
    private String nombre;
    private int vida;

    // 1. Constructor Completo
    public Personaje(String nombre, int vida) {
        this.nombre = nombre;
        this.setVida(vida); // Buena practica! Usar el setter para validar al nac
    }

    // 2. Constructor Rapido (Valores por defecto)
    public Personaje(String nombre) {
        this.nombre = nombre;
        this.vida = 100; // Asume vida 100
    }

    // 3. Constructor Copia (Clona un objeto existente)
    public Personaje(Personaje p) {
        this.nombre = p.nombre;
        this.vida = p.vida;
    }
}
```

6. Miembros Estaticos (static) y Constantes

- **Atributo static:** No pertenece a un objeto concreto, pertenece a la Clase (es global y compartido). Existe una sola copia en memoria. Si un objeto lo cambia, cambia para todos. Se accede llamando a la clase: Personaje.getTotalPersonajes().
- **Constantes Estaticas (public static final):** Valores fijos que nunca cambian y son compartidos por todos (ej. la vida maxima permitida en el juego).

7. Ejemplo Integrador de la Unidad 5: Gestion de una Libreria

Este ejemplo reúne todos los conceptos basicos: creacion de clases, atributos private (encapsulamiento), constructores sobrecargados, setters con validacion y uso de objetos en un programa principal (Main).

Clase Libro (El Molde):

```
package Libreria;

public class Libro {
    // 1. Encapsulamiento: Atributos privados
    private String ISBN;
    private String titulo;
    private String autor;
    private int numPaginas;

    // 2. Constructor Completo
    public Libro(String ISBN, String titulo, String autor, int numPaginas) {
```

```

        this.ISBN = ISBN;
        this.titulo = titulo;
        this.autor = autor;
        this.setNumPaginas(numPaginas); // Usamos el setter para validar al nacer
    }

    // 3. Constructor de Copia (Sobrecarga de constructores)
    public Libro(Libro l) {
        this(l.ISBN, l.titulo, l.autor, l.numPaginas); // Reutiliza el constructo
    }

    // 4. Getters y Setters (Interfaz publica)
    public int getNumPaginas() { return numPaginas; }

    public void setNumPaginas(int numPaginas) {
        // Validacion: Si introducen paginas negativas, lo deja en 0
        this.numPaginas = (numPaginas < 0) ? 0 : numPaginas;
    }

    public String getAutor() { return this.autor; }
    public void setAutor(String autor) { this.autor = autor; }

    public String getISBN() { return this.ISBN; }
    public void setISBN(String ISBN) { this.ISBN = ISBN; }

    public String getTitulo() { return this.titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }

    // 5. Sobreescritura del metodo toString para imprimir el objeto
    @Override
    public String toString() {
        return "[" + this.ISBN + "] " + this.titulo + " - " + this.autor + " (" +
    }
}

```

Uso de los Objetos en el Main:

```

package Libreria;
import java.util.ArrayList;
import java.util.Scanner;

public class MainLibreria {
    public static void main(String[] args) {
        // Coleccion para guardar multiples objetos Libro
        ArrayList<Libro> libreria = new ArrayList<>();

        // Uso del constructor y creacion de instancias con new
        Libro libro1 = new Libro("1234A", "El Senor de los Anillos", "Tolkien", 1);
        libreria.add(libro1);

        // Intentando asignar un valor invalido (el setter lo corrige)
        Libro libro2 = new Libro("5678B", "El Principito", "Saint-Exupery", -50);
        libreria.add(libro2); // Se guardara con 0 paginas gracias al setter

        // Ver la libreria (se llama automaticamente a toString())
        for (Libro l : libreria) {
            System.out.println(l);
        }
    }
}

```

```

    }
}
}

```

UNIDAD 6: POO Avanzada

1. Clases Compuestas (Composicion) -> Relacion "TIENE-UN"

El software es como un Lego. Creamos clases complejas usando otras clases como "piezas" o atributos. **Delegacion:** La clase principal no hace todo el trabajo, "delega" tareas en sus componentes.

```

// PIEZA: Arma
public class Arma {
    private int dano;
    public int getDano() { return dano; }
}

// CONJUNTO: Personaje (TIENE un Arma)
public class Personaje {
    private Arma armaEquipada; // Atributo que es otro objeto

    public void atacar() {
        // El personaje no calcula el dano, DELEGA en el arma
        System.out.println("Dano: " + this.armaEquipada.getDano());
    }
}

```

2. Herencia -> Relacion "ES-UN"

Permite crear clases nuevas (Hijas/Subclases) basadas en clases existentes (Padres/Superclases) para reutilizar codigo.

- **extends:** Palabra reservada. *Nota:* En Java NO existe la herencia multiple de clases (solo puedes tener 1 padre).
- **protected:** Modificador de acceso perfecto para herencia. Hace que un atributo sea privado para el exterior, pero visible para las clases hijas.
- **Sobreescritura (@Override):** El hijo hereda un metodo del padre, pero le cambia el codigo para hacerlo a su manera.

REGLA DE LOS CIMIENTOS (super): Cuando una clase hija tiene un constructor, la primera linea debe ser obligatoriamente super(...) para construir primero la parte del padre.

```

// PADRE
public class Personaje {
    protected String nombre; // Protected: El hijo podra verlo
    protected int vida;
}

```

```

    public Personaje(String nombre, int vida) {
        this.nombre = nombre;
        this.vida = vida;
    }
    public void saludar() { System.out.println("Hola generico"); }
}

// HIJO (Guerrero ES UN Personaje)
public class Guerrero extends Personaje {
    private int furia;

    public Guerrero(String nombre, int vida, int furia) {
        super(nombre, vida); // 1. OBLIGATORIO: Construir al padre
        this.furia = furia; // 2. Construir lo del hijo
    }

    @Override
    public void saludar() {
        System.out.println("SOY " + this.nombre + " Y GRITO!"); // Metodo reescri
    }
}

```

3. Clases Abstractas (abstract)

Son conceptos puros que **no se pueden instanciar** (new Figura() da error). Solo sirven para que otras clases hereden de ellas.

REGLA DE ORO DE LA ABSTRACCION: Si heredas de una clase abstracta, estas **OBLIGADO** a implementar (darle código) a todos sus métodos abstractos. Si no lo haces, tu también serás abstracto.

- **Método Abstracto:** Termina en ";" en lugar de "{}". Promete que los hijos sabrán hacerlo, pero el padre no sabe cómo.
- *(Nota: Lo opuesto a abstract es final. Una clase final no puede tener hijos, y un método final no puede ser sobrescrito).*

```

public abstract class Figura {
    // Obliga a los hijos a crear este metodo
    public abstract double calcularArea();
}

public class Rectangulo extends Figura {
    private double base, altura;

    @Override // Cumpliendo el contrato obligatorio
    public double calcularArea() { return base * altura; }
}

```

4. Interfaces (implements)

Definen una **CAPACIDAD** (lo que sabes hacer), no tu identidad. Es un contrato horizontal.

- Una clase puede heredar de 1 solo padre, pero **puede implementar MULTIPLES interfaces** (separadas por comas).
- Todos los metodos de una interfaz son abstractos por defecto (solo definen el "que", las clases concretas definen el "como").

```
// LA CAPACIDAD (El contrato)
public interface Localizable {
    double getLatitud();
    double getLongitud();
}

// UNA CLASE CUALQUIERA QUE FIRMA EL CONTRATO
public class Perro implements Localizable {
    @Override
    public double getLatitud() { return 39.5; }
    @Override
    public double getLongitud() { return -0.2; }
}
```

5. Polimorfismo ("Muchas formas")

Es la capacidad de tratar a objetos diferentes (hijos) de forma generica (como si fueran su padre o su interfaz).

- **La Caja Magica:** Guardas un Guerrero dentro de una variable de tipo Personaje. Java usa la **Ligadura Dinamica** para decidir en tiempo real que, si le dices "ataca", usara el ataque del Guerrero y no el del generico.
- **Casting y instanceof:** Si tienes un objeto en una variable generica y necesitas usar un metodo exclusivo suyo (ej. gritoDeGuerra() del Guerrero), primero debes usar instanceof como "rayos X" para asegurar que es, y luego hacer un "Downcasting".

6. Igualdad y Orden (Comparadores)

Por defecto, el operador "==" en Java solo mira si dos variables apuntan a la *misma direccion de memoria*. Para comparar el *contenido real* (ej. saber si dos objetos son el mismo usuario porque tienen el mismo DNI), debemos sobrescribir metodos.

Igualdad de Contenido (equals y hashCode)

EL CONTRATO (REGLA DE ORO ABSOLUTA): Siempre que sobrescribas equals para decir que dos objetos son iguales, **DEBES** sobrescribir hashCode obligatoriamente. ¿Por que? Porque colecciones de Java como HashSet o HashMap usan primero el hashCode (un numero identificativo rapido) para buscar. Si dos objetos son "iguales" segun tu equals pero no actualizaste el hashCode para que devuelva el mismo numero, las colecciones fallaran por completo y pensaran que son distintos.

Orden Natural (Comparable)

Si quieres ordenar una lista de objetos (ej. con `Collections.sort()`), la clase debe implementar la interfaz `Comparable` y su metodo `compareTo`.

7. Ejemplo Integrador de la Unidad 6: Juego de Rol (RPG) Completo

A continuacion, el codigo integro de todo el sistema del Juego de Rol. Este ejemplo reúne todos los conceptos avanzados: **Clases Abstractas, Interfaces, Herencia, Composicion, Polimorfismo y Sobreescritura** (`equals`, `hashCode` y `compareTo`).

1. Interfaz Atacable (Capacidad):

```
package U6.juego_rol.model;

public interface Atacable {
    void recibirDano(int cantidad);

    String getNombre();

    boolean estaVivo();
}
```

2. Clase Item (Composicion):

```
package U6.juego_rol.model;

public class Item {
    private String nombre;
    private int aumentoAtaque;
    private int aumentoDefensa;

    public Item(String nombre, int aumentoAtaque, int aumentoDefensa) {
        this.nombre = nombre;
        this.aumentoAtaque = aumentoAtaque;
        this.aumentoDefensa = aumentoDefensa;
    }

    public String getNombre() {
        return nombre;
    }

    public int getAumentoAtaque() {
        return aumentoAtaque;
    }

    public int getAumentoDefensa() {
        return aumentoDefensa;
    }

    @Override
    public String toString() {
        return String.format("%s (Atq: +%d, Def: +%d)", nombre, aumentoAtaque, au
    }
}
```

}

3. Clase Abstracta Base (Personaje):

```

package U6.juego_rol.model;

import java.util.ArrayList;
import java.util.Objects;

public abstract class Personaje implements Atacable, Comparable<Personaje> {
    protected String nombre;
    protected int vida;
    protected int nivel;
    protected int agilidad;
    protected int resistencia;
    protected ArrayList<Item> mochila;

    public Personaje(String nombre, int vida, int nivel, int agilidad, int resist
        this.nombre = nombre;
        this.vida = vida;
        this.nivel = nivel;
        this.agilidad = agilidad;
        this.resistencia = resistencia;
        this.mochila = new ArrayList<>();
    }

    public abstract void atacar(Personaje objetivo);

    public void cogerItem(Item i) {
        this.mochila.add(i);
        System.out.println(nombre + " ha cogido: " + i.getNombre());
    }

    protected int getBonusAtaque() {
        int total = 0;
        for (Item i : mochila) {
            total += i.getAumentoAtaque();
        }
        return total;
    }

    protected int getBonusDefensa() {
        int total = 0;
        for (Item i : mochila) {
            total += i.getAumentoDefensa();
        }
        return total;
    }

    @Override
    public void recibirDano(int dano) {
        int danoReal = dano - (resistencia + getBonusDefensa());
        if (danoReal < 0)
            danoReal = 0;

        this.vida -= danoReal;
        System.out.println(nombre + " recibe " + danoReal + " de daño. Vida restc

```

```

    }

    @Override
    public String getNombre() {
        return nombre;
    }

    @Override
    public boolean estaVivo() {
        return vida > 0;
    }

    @Override
    public int compareTo(Personaje o) {
        // Orden descendente por agilidad
        return Integer.compare(o.agilidad, this.agilidad);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (o == null || getClass() != o.getClass())
            return false;
        Personaje personaje = (Personaje) o;
        return Objects.equals(nombre, personaje.nombre);
    }

    @Override
    public int hashCode() {
        return Objects.hash(nombre);
    }

    @Override
    public String toString() {
        return String.format("%s (%s) | Nivel: %d | Vida: %d | Agilidad: %d | Resistencia: %d",
            nombre, getClass().getSimpleName(), nivel, vida, agilidad, resist);
    }

    // Getters for RPG logic
    public int getVida() {
        return vida;
    }

    public int getNivel() {
        return nivel;
    }

    public int getAgilidad() {
        return agilidad;
    }
}

```

4. Clases Concretas (Guerrero, Mago, Orco):

```
package U6.juego_rol.model;
```

```
public class Guerrero extends Personaje {
    private int furia;

    public Guerrero(String nombre, int vida, int nivel, int agilidad, int resiste
        super(nombre, vida, nivel, agilidad, resistencia);
        this.furia = furia;
    }

    @Override
    public void atacar(Personaje objetivo) {
        int danoBase = nivel * 2 + furia + getBonusAtaque();
        if (vida < 10) {
            System.out.println(nombre + " entra en estado BERSERKER!");
            danoBase *= 2;
        }
        System.out.println(nombre + " ataca con espada a " + objetivo.getNombre()
            objetivo.recibirDano(danoBase);
        }

    @Override
    public String toString() {
        return super.toString() + " | Furia: " + furia;
    }
}
```

```
package U6.juego_rol.model;
```

```
public class Mago extends Personaje {
    private int mana;

    public Mago(String nombre, int vida, int nivel, int agilidad, int resistenc
        super(nombre, vida, nivel, agilidad, resistencia);
        this.mana = mana;
    }

    @Override
    public void atacar(Personaje objetivo) {
        if (mana < 10) {
            System.out.println(nombre + " no tiene suficiente maná... Recuperand
            mana += 5;
            return;
        }

        int danoBase = nivel * nivel + mana + getBonusAtaque();
        mana -= 10;
        System.out.println(nombre + " lanza un hechizo a " + objetivo.getNombre()
            objetivo.recibirDano(danoBase);
        }

    @Override
    public String toString() {
        return super.toString() + " | Maná: " + mana;
    }
}
```

```

package U6.juego_rol.model;

public class Orco extends Personaje {
    private int rabia;

    public Orco(String nombre, int vida, int nivel, int agilidad, int resistencia) {
        super(nombre, vida, nivel, agilidad, resistencia);
        this.rabia = rabia;
    }

    @Override
    public void atacar(Personaje objetivo) {

        int danoPuro = nivel * 3 + rabia + getBonusAtaque();
        System.out.println(nombre + " lanza un ataque brutal que ignora defensas

        objetivo.recibirDano(danoPuro + objetivo.resistencia + objetivo.getBonusD
    }

    @Override
    public String toString() {
        return super.toString() + " | Rabia: " + rabia;
    }
}

```

5. Clase Gremio (Gestion de Colecciones y Orden):

```

package U6.juego_rol.model;

import java.util.ArrayList;
import java.util.Collections;

public class Gremio {
    private String nombre;
    private ArrayList<Personaje> miembros;

    public Gremio(String nombre) {
        this.nombre = nombre;
        this.miembros = new ArrayList<>();
    }

    public void reclutar(Personaje p) {
        if (!miembros.contains(p)) {
            miembros.add(p);
            System.out.println(p.getNombre() + " se ha unido al gremio " + nombre
        } else {
            System.out.println(p.getNombre() + " ya está en el gremio.");
        }
    }

    public void expulsar(Personaje p) {
        if (miembros.remove(p)) {
            System.out.println(p.getNombre() + " ha sido expulsado del gremio " +
        } else {
            System.out.println(p.getNombre() + " no pertenece a este gremio.");
        }
    }
}

```

```

    }

    public void mostrarMiembros() {
        System.out.println("--- Miembros del Gremio: " + nombre + " ---");
        // Ordenación natural por agilidad (descendente) definida en Personaje.cc
        Collections.sort(miembros);
        for (Personaje p : miembros) {
            System.out.println(p);
        }
    }

    public ArrayList<Personaje> getMiembros() {
        return miembros;
    }
}

```

6. El Programa Principal (Main):

```

package U6.juego_rol.main;

import U6.juego_rol.model.*;

import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    private static ArrayList<Personaje> personajesSueltos = new ArrayList<>();
    private static Gremio miGremio = null;
    private static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        int opcion;
        do {
            System.out.println("\n--- BIENVENIDO A LA BATALLA POR EL TRONO DE CÓC");
            System.out.println("1. Alta de Personaje");
            System.out.println("2. Crear Gremio");
            System.out.println("3. Gestión de Gremio (Añadir/Echar)");
            System.out.println("4. Entrenar (Combate)");
            System.out.println("5. Mostrar Estado");
            System.out.println("0. Salir");
            System.out.print("Elige una opción: ");
            opcion = sc.nextInt();
            sc.nextLine(); // Limpiar buffer

            switch (opcion) {
                case 1:
                    altaPersonaje();
                    break;
                case 2:
                    crearGremio();
                    break;
                case 3:
                    gestionGremio();
                    break;
                case 4:
                    entrenar();
                    break;
            }
        } while (opcion != 0);
    }
}

```

```
        case 5:
            mostrarEstado();
            break;
        case 0:
            System.out.println("¡Hasta la próxima aventurero!");
            break;
        default:
            System.out.println("Opción no válida.");
            break;
    }
} while (opcion != 0);
}

private static void altaPersonaje() {
    System.out.print("Nombre: ");
    String nombre = sc.nextLine();
    System.out.println("Tipo: 1. Guerrero | 2. Mago | 3. Orco");
    int tipo = sc.nextInt();
    System.out.print("Nivel: ");
    int nivel = sc.nextInt();
    System.out.print("Vida: ");
    int vida = sc.nextInt();
    System.out.print("Agilidad: ");
    int agilidad = sc.nextInt();
    System.out.print("Resistencia: ");
    int resistencia = sc.nextInt();

    Personaje p = null;
    switch (tipo) {
        case 1:
            System.out.print("Furia: ");
            p = new Guerrero(nombre, vida, nivel, agilidad, resistencia, sc.nextLine());
            break;
        case 2:
            System.out.print("Maná: ");
            p = new Mago(nombre, vida, nivel, agilidad, resistencia, sc.nextLine());
            break;
        case 3:
            System.out.print("Rabia: ");
            p = new Orco(nombre, vida, nivel, agilidad, resistencia, sc.nextLine());
            break;
        default:
            p = null;
            break;
    }

    if (p != null) {
        personajesSuelto.add(p);
        System.out.println("Personaje creado con éxito.");
    }
}

private static void crearGremio() {
    System.out.print("Nombre del Gremio: ");
    String nombre = sc.nextLine();
    miGremio = new Gremio(nombre);
    System.out.println("Gremio '" + nombre + "' creado.");
}

private static void gestionGremio() {
    if (miGremio == null) {
```

```

        System.out.println("Primero debes crear un gremio.");
        return;
    }
    System.out.println("1. Reclutar personaje suelto");
    System.out.println("2. Expulsar del gremio");
    int opt = sc.nextInt();
    sc.nextLine();

    if (opt == 1) {
        if (personajesSuelos.isEmpty()) {
            System.out.println("No hay personajes sueltos disponibles.");
            return;
        }
        for (int i = 0; i < personajesSuelos.size(); i++) {
            System.out.println(i + ". " + personajesSuelos.get(i));
        }
        System.out.print("Elige índice: ");
        int idx = sc.nextInt();
        if (idx >= 0 && idx < personajesSuelos.size()) {
            Personaje p = personajesSuelos.remove(idx);
            miGremio.reclutar(p);
        }
    } else if (opt == 2) {
        ArrayList<Personaje> miembros = miGremio.getMembros();
        if (miembros.isEmpty()) {
            System.out.println("El gremio está vacío.");
            return;
        }
        for (int i = 0; i < miembros.size(); i++) {
            System.out.println(i + ". " + miembros.get(i));
        }
        System.out.print("Elige índice: ");
        int idx = sc.nextInt();
        if (idx >= 0 && idx < miembros.size()) {
            Personaje p = miembros.get(idx);
            miGremio.expulsar(p);
            personajesSuelos.add(p);
        }
    }
}

private static void entrenar() {
    if (miGremio == null || miGremio.getMembros().size() < 2) {
        System.out.println("Necesitas al menos 2 personajes en el gremio para");
        return;
    }

    ArrayList<Personaje> combatientes = new ArrayList<>(miGremio.getMembros()
    // El enunciado sugiere elegir dos.
    System.out.println("Elige primer combatiente:");
    for (int i = 0; i < combatientes.size(); i++) {
        System.out.println(i + ". " + combatientes.get(i).getNombre());
    }
    int c1Idx = sc.nextInt();

    System.out.println("Elige segundo combatiente:");
    for (int i = 0; i < combatientes.size(); i++) {
        System.out.println(i + ". " + combatientes.get(i).getNombre());
    }
    int c2Idx = sc.nextInt();

```

```
if (c1Idx == c2Idx) {
    System.out.println("¡No pueden luchar contra sí mismos!");
    return;
}

Personaje p1 = combatientes.get(c1Idx);
Personaje p2 = combatientes.get(c2Idx);

System.out.println("\n--- ¡COMIENZA EL DUELO! ---");
System.out.println(p1.getNombre() + " VS " + p2.getNombre());

int turno = 1;
while (p1.estaVivo() && p2.estaVivo()) {
    System.out.println("\n--- Turno " + turno + " ---");
    // Decidir orden por agilidad
    if (p1.getAgilidad() >= p2.getAgilidad()) {
        p1.atacar(p2);
        if (p2.estaVivo())
            p2.atacar(p1);
    } else {
        p2.atacar(p1);
        if (p1.estaVivo())
            p1.atacar(p2);
    }
    turno++;
    if (turno > 100) { // Seguro por si acaso
        System.out.println("¡Es un empate por agotamiento!");
        break;
    }
}

if (p1.estaVivo() && !p2.estaVivo())
    System.out.println("\n¡GANADOR: " + p1.getNombre() + "!");
else if (p2.estaVivo() && !p1.estaVivo())
    System.out.println("\n¡GANADOR: " + p2.getNombre() + "!");
}

private static void mostrarEstado() {
    if (miGremio != null) {
        miGremio.mostrarMiembros();
    }
    if (!personajesSuelto.isEmpty()) {
        System.out.println("\n--- Personajes Sin Gremio ---");
        for (Personaje p : personajesSuelto) {
```